
530 PROJECT FINAL REFLECTION

TCP Flood and Port Scan Detector: A Wireshark-Inspired Network Security Tool

Mahesh Babu Tasil

Spring 2025

PROJECT DESCRIPTION

This project aims to develop a Python-based application, "TCP Flood Detector," that monitors TCP/IP packets across all network interfaces to detect SYN flood attacks and port scans, two prevalent network security threats. The application captures packets in real-time using the Scapy library, displays them in a Wireshark-like interface with a tabular format (columns for timestamp, interface, source IP, source port, destination IP, destination port, and flags), and alerts users to potential attacks based on predefined thresholds (e.g., 100 SYN packets in 10 seconds for floods, 10 unique ports for scans). The tool addresses the critical issue of denial-of-service (DoS) attacks and reconnaissance attempts, which are significant in network security as they can disrupt services or identify vulnerabilities for further exploitation. By providing real-time monitoring and visualization, the project aligns with the "programming tool" category, offering network administrators a practical utility to enhance network security. The project is achievable within the Spring 2025 semester.

TARGET AUDIENCE

The target audience includes network administrators, security analysts, and IT professionals responsible for monitoring and securing organizational networks. This group typically manages network traffic, responds to security incidents, and seeks tools to detect and mitigate threats like DoS attacks and port scans.

AUDIENCE SKILL LEVEL

The target audience is expected to have an intermediate to advanced skill level in network security. They should be familiar with the TCP/IP model, network layers (especially the transport layer), and common attack vectors like SYN floods. Prior knowledge of packet analysis tools (e.g., Wireshark) and basic Python programming is beneficial but not required, as the tool provides a user-friendly GUI.

LEARNING OBJECTIVES

After a participant views or interacts with the project, they should be able to:

- Objective 1** – Identify and interpret TCP/IP packet details (e.g., source/destination IPs, ports, flags) in a tabular format, like Wireshark, to understand network traffic patterns.
- Objective 2** – Detect SYN flood attacks by recognizing when the number of SYN packets exceeds a threshold, enabling timely mitigation of DoS threats.

Objective 3 – Recognize port scanning attempts by observing multiple unique port connections from a single IP, allowing proactive defense against reconnaissance attacks.

PROJECT IMPACT

This tool significantly enhances understanding of TCP-level attacks by providing an accessible, GUI-based detection system that visually explains how SYN floods and port scans manifest in real time. It is particularly impactful for educational use, giving learners a hands-on way to simulate attacks and immediately see their network effects. The project fills a gap by combining practical attack detection with an easy-to-use interface that includes color coding, tooltips, and filtering—features inspired by Wireshark but tailored for simplicity and learning. By providing a live packet graph, detailed packet views, and .pcap support, the project turns theoretical concepts into practical, observable phenomena.

EXPECTED OUTCOMES, EVALUATION, AND ASSESSMENT

- **Expected Deliverables:**
 - A Python-based application that captures TCP/IP packets, detects SYN floods and port scans, and displays packets in a QWidget with columns (Timestamp, Interface, Src IP, Src Port, Dst IP, Dst Port, Flags).
 - A user guide (in the Help section) explaining usage, thresholds, and network setup.
 - A .pcap export feature for further analysis in Wireshark.
- **Alignment with Objectives:**
 - The packet table aligns with Objective 1 (interpret packet details).
 - Flood and port scan detection aligns with Objectives 2 and 3 (detect attacks).
- **Evaluation Criteria and Methods:**
 - **Functionality:** The tool must capture packets, detect attacks, and stop cleanly. I will test by simulating 200 SYN packets (flood) and scanning 15 ports (port scan); stop functionality must complete within 5 seconds.
 - **Usability:** The GUI must be intuitive, with clear packet display and alerts. I will evaluate usability through feedback from 2-3 peers on ease of use.
 - **Stability:** The app must not crash during 30 minutes of continuous monitoring. I will assess stability by running the tool with real network traffic and logging errors.
- **Metrics/Indicators:**
 - Detection accuracy: 100% detection rate for simulated attacks (200 SYN packets, 15 ports scanned).
 - Stop time: Less than 5 seconds to stop all threads, measured via logs.
 - User satisfaction: At least 80% positive feedback on usability from peer reviews.

PROJECT EVALUATION AND OUTCOMES

Since the last submission, new features were added: advanced filtering (by TCP flags and source IP), tooltip headers, a reset layout button, and PCAP import functionality. These additions addressed instructor feedback and improved user understanding and usability. The tool successfully met its objectives: students can now identify TCP flags, observe attack patterns, and simulate both SYN floods and port scans. All core outcomes were achieved, including live visualization and alerting. A strength of the project is its user-friendly design and accurate detection. A minor weakness is that large PCAP files load slowly, which may be optimized in the future.

PERSONAL IMPACT

I chose this project because of my interest in real-time network security and my desire to build a practical tool that brings abstract attack concepts to life. Seeing attacks visualized in real time deepened my appreciation for network monitoring and defense techniques.

PERSONAL LEARNING GOALS

Through this project, I aim to deepen my understanding of the TCP/IP model, particularly the transport layer, and enhance my skills in network security and Python programming. I want to master packet analysis using Scapy, learning how to parse and interpret TCP packets effectively to detect threats like SYN floods and port scans. Additionally, I hope to improve my GUI development skills with PyQt5, creating a user-friendly interface that rivals professional tools like Wireshark. A key challenge I'm tackling is the stop functionality issue, which will teach me advanced threading and resource management in Python. This project will contribute to my growth by providing hands-on experience with real-world network security challenges, preparing me for a career in cybersecurity where such skills are critical. I also aim to improve my problem-solving and debugging abilities, ensuring I can deliver a robust, production-ready tool by the end of the semester.

SKILLS AND KNOWLEDGE GAINED

This project significantly expanded my skills in Python networking, GUI development, and threading. I learned advanced Scapy functions for real-time packet capture, implemented multithreading safely within PyQt5, and tackled GUI design challenges like live graphing and dynamic table updates. I also deepened my understanding of TCP/IP mechanics, attack patterns, and how security tools like Wireshark organize data.

LESSONS LEARNED

One major lesson was managing Scapy's blocking sniff function alongside the PyQt5 event loop, which required threading and careful synchronization. Testing attacks locally (e.g., hping3, nmap) gave me real-world insight into packet flows. Time management was critical when balancing coding and documentation. If I were to restart, I would design the GUI layout earlier to avoid later refactoring.

FUTURE IMPACT

Future improvements could include multi-protocol support (adding UDP/ICMP), deeper packet inspection, and automated reporting of detected attacks. I'm also interested in integrating ML-based anomaly detection. This project has strengthened my interest in cybersecurity tool development and may guide future career choices toward network defense roles.

EXPLANATION OF SUBMITTED MATERIALS

- **wireshark_style_detector_with_pcap.txt:** Main application code, fully commented, with Scapy sniffers, PyQt5 GUI, and detection logic.
- **README.txt:** Setup guide, dependencies, usage instructions, simulation commands.
- **educational_walkthrough_final_expanded.docx:** Expanded guide explaining TCP/IP attacks, detection logic, simulation labs, and teaching use cases.
- **530 Project Content Submission - Mahesh Babu Tasil.pdf:** Full documentation of research, content, and development details.

PROJECT CONTENT DELIVERABLES

Item 1

- Item Description: Complete source code of the TCP Flood and Port Scan Detector tool with GUI, real-time detection, filtering, graphing, and export/import functionality.
- File name/path/link: wireshark_style_detector_with_pcap.txt (main code file, converted from .py)
- Relevant information: Includes all UI features, detection logic, threading, Scapy sniffers, .pcap import/export, and filter integration. Fully functional when dependencies are installed.

Item 2

- Item Description: Educational documentation including attack explanation, TCP/IP fundamentals, and GUI feature walkthrough
- File name/path/link: educational_walkthrough.pdf
- Relevant information: Supports learning outcomes by explaining SYN floods, port scans, TCP handshake, and GUI functionality in simple terms. Can be integrated into a lab or demo

Item 3

- Item Description: README file
- File name/path/link: README.txt
- Relevant information: Includes installation instructions, tool usage, packet capture requirements, and simulation guide

SOURCE CODE AND READMEs

Item 1

- Item Description: Main Python application with full detection logic and Wireshark-style GUI interface.
- File name/path/link: wireshark_style_detector_with_pcap.txt (converted from .py)
- Relevant information: Contains inline comments for clarity, structured with distinct methods for sniffing, filtering, and GUI management. Submitted as .txt as required. This is the primary executable file.

Item 2

- Item Description: README documentation
- File name/path/link: README.txt
- Relevant information: Explains setup (Python 3.9+, scapy, pyqt5, pyqtgraph, Npcap), includes a run command example, and outlines basic usage of the interface. Mentions optional tools for simulation (nmap, hping3).

DEPENDENCIES, RESOURCES & TOOLS

Python Version: Python 3.9+

Libraries/Packages:

- scapy (for packet capturing and parsing)
- PyQt5 (for GUI interface)
- pyqtgraph (for live graph plotting)

Tools:

- Npcap (required for Scapy to capture packets on Windows)
- Visual Studio Code (used for development)
- Git (for version control)

Hardware:

- Windows machine with administrative privileges (required to install Npcap)

Optional Tools for Testing:

- nmap (to simulate port scans)
- hping3 (to simulate SYN flood attacks)

Offline Analysis:

- Ability to load .pcap files created using Wireshark or similar tools

REFERENCES

- [1] A. Liska and K. Stowe, *Cybersecurity Fundamentals: A Real-World Perspective*, 2nd ed. Boston, MA, USA: Jones & Bartlett Learning, 2023.
- [2] J. Smith, "Network Traffic Analysis with Scapy," in *Journal of Network Security*, vol. 25, no. 3, pp. 4552, Mar. 2023.
- PyQt5 QWidget Docs: <https://doc.qt.io/qtforpython/>
- Wireshark User Guide: <https://www.wireshark.org/docs/>